

Lab 3A: Custom Commands — Your Team's Playbook

Duration: 45 minutes

Difficulty: Intermediate

Continues from: Day 2 Business Dashboard project

© 2026 AIA Copilot — Instructor-Led Training Material

Overview

Yesterday you built a complete dashboard application. Today you'll make Claude Code work the way YOUR team works. Custom commands turn repetitive workflows into one-word instructions that anyone on your team can run.

What You'll Build

- A `/review` command that runs your team's code review checklist
 - A `/doc` command that generates API documentation from any endpoint
 - A `/refactor` command that analyzes and improves code quality
 - A `/deploy-check` command that runs pre-deployment verification
 - A `/changelog` command that summarizes recent git activity
 - Iterate and refine commands based on real output
-

Setup

Open your Day 2 dashboard project:

```
cd business-dashboard
claude
```

Phase 1: Your First Custom Command

Create the commands directory:

```
Create a .claude/commands directory for custom slash commands
```

Now create your first command. Tell Claude:

```
Create a file .claude/commands/review.md with this content:

---
description: Run a code review checklist on a file or directory
---

Review $ARGUMENTS using this checklist:

1. Error handling – are all errors caught and handled gracefully?
2. Input validation – is all user input validated before use?
3. Naming – are variables and functions clearly named?
4. DRY – is there any duplicated logic that should be extracted?
5. Security – any hardcoded secrets, SQL injection risks, or XSS vulnerabilities?
6. Testing – does test coverage exist for the main code paths?
7. Documentation – are exported functions documented?

For each item, report PASS, FAIL, or NEEDS ATTENTION with a one-line explanation.
End with an overall assessment and the top 3 improvements to make.
```

Test it immediately:

```
/review server.js
```

Does the output follow your checklist? Is it actionable?

Now test on a different file:

```
/review src/routes/tasks.js
```

Notice: same checklist, same format, different file. That's consistency you can't get from ad-hoc prompts.

Phase 2: Iterate on Your Command

The first version of a command is never perfect. Review what `/review` produced and improve it.

```
The /review command output was too verbose. Update .claude/commands/review.md
to add this instruction at the end:
```

```
"Keep each checklist item to ONE line. No explanations longer than 15 words.
The overall assessment should be 2-3 sentences maximum."
```

Test the updated version:

```
/review server.js
```

Compare to the first run. Tighter? More actionable? This iteration loop — create, test, refine — is how you build commands your team will actually use.

Try one more refinement:

```
Add a severity rating to each checklist item in /review:
[CRITICAL], [MODERATE], or [LOW].
Only expand on items rated CRITICAL.
```

Test again. The command should now produce a scannable report that highlights what actually matters.

Phase 3: API Documentation Command

Create a command that generates consistent API documentation:

Create `.claude/commands/doc.md` that generates API documentation for `$ARGUMENTS`.

Include these sections in order:

- Endpoint Summary (one line)
- HTTP method, path, and authentication requirements
- Request format with JSON body example (if applicable)
- Query parameters table (name, type, required, description)
- Success response with JSON example
- Error responses table (status code, condition, response body)
- curl example for each HTTP method the endpoint supports

Format as clean markdown. Use tables where appropriate.

Test it:

```
/doc src/routes/tasks.js
```

Now test on a different file:

```
/doc src/routes/metrics.js
```

Compare the two outputs. The format should be identical — only the content changes. If the format drifts between files, refine the command to be more prescriptive.

Now generate docs for all your endpoints at once:

```
/doc src/routes/
```

This is a real deliverable. Save the output:

```
Save the API documentation you just generated to docs/API.md
```

Phase 4: Refactoring Command

```
Create .claude/commands/refactor.md that analyzes $ARGUMENTS for:
- Functions longer than 20 lines
- Nesting deeper than 3 levels
- Duplicated code patterns
- Missing async/await (using callbacks instead)
- Magic numbers or hardcoded strings
- Missing error handling on async operations
- Functions doing more than one thing (Single Responsibility violations)

For each finding:
1. What: the specific issue (file and line)
2. Why: one sentence on the impact
3. How: concrete code showing the fix
Priority: HIGH / MEDIUM / LOW

Start with HIGH priority items. Stop after 5 findings unless asked for more.
```

Test it:

```
/refactor server.js
```

Does Claude find real issues? Now implement the fixes:

```
Implement all HIGH priority refactoring suggestions you just identified.
Run the tests after each change to make sure nothing breaks.
```

Watch Claude work through each fix, testing as it goes. This is the workflow: `/refactor` identifies → you review → Claude implements → tests verify.

Phase 5: Pre-Deploy Verification

This command is your safety net before shipping:

Create `.claude/commands/deploy-check.md` that runs a pre-deployment checklist:

1. Run the test suite – report pass/fail count
2. Search for `console.log` statements in `src/` (not in test files)
3. Search for `TODO` and `FIXME` comments in `src/`
4. Check that all API endpoints return proper error responses (not raw stack traces)
5. Verify no hardcoded localhost URLs exist in `src/`
6. Check for any `.env` or credential files that might be staged in git
7. Verify `package.json` has no missing dependencies (`npm ls --depth=0`)

Report each item as `PASS` or `FAIL` with a one-line detail.

At the end, give a clear `GO` or `NO-GO` verdict.

If ANY critical item fails, output: `"❌ DO NOT DEPLOY"` with the failures listed.

Test it:

```
/deploy-check
```

Fix any failures Claude finds:

```
Fix all the FAIL items from the deploy check, then run /deploy-check again
```

Keep iterating until you get a clean `GO` verdict. This is exactly how you'd use it in a real CI/CD workflow.

Phase 6: Changelog Command

One more practical command:

```
Create .claude/commands/changelog.md that generates a changelog from recent git history.
```

```
Analyze $ARGUMENTS (a number of commits, default 10) and produce:
```

```
## Changelog
```

```
### Features
```

```
- list new features with commit references
```

```
### Fixes
```

```
- list bug fixes with commit references
```

```
### Refactoring
```

```
- list refactoring changes
```

```
### Other
```

```
- anything that doesn't fit above
```

```
Group by category, not by commit. If multiple commits relate to the same feature, combine them into one entry. Use conventional commit messages as hints for categorization.
```

Test it:

```
/changelog 20
```

Phase 7: Share Your Commands

Your commands are ready for the team. Commit them:

```
Stage and commit the .claude/commands/ directory with the message  
"feat: add team workflow commands (review, doc, refactor, deploy-check, changelog)"
```

When any team member pulls this commit, they get all five commands in their  menu. Zero onboarding — the team's tribal knowledge is encoded and shared.

Verify the commands are committed:

```
Show me the git log for the last commit and list the files included
```

What You Learned

- Custom commands live in `.claude/commands/` as markdown files
- `$ARGUMENTS` gets replaced with whatever follows the command
- The iterate loop: create → test → refine. First version is never final.
- Commands enforce consistency — same checklist, same format, every time
- Commands are committed to git — your whole team gets them instantly
- Real power: encoding tribal knowledge into repeatable one-word workflows

The Rule

If you've typed the same kind of instruction three times, make it a command.

Commands You Built

Command	Purpose
<code>/review</code>	Code review checklist with severity ratings
<code>/doc</code>	Consistent API documentation generation
<code>/refactor</code>	Code quality analysis with prioritized fixes
<code>/deploy-check</code>	Pre-deployment verification (GO / NO-GO)
<code>/changelog</code>	Git history → categorized changelog

Keep your dashboard project open — Lab 3B builds on it.

